

System Design Document

OutYah

Jamaica Outing Discovery Platform

Week 3 Deliverable (July 13 - July 17, 2026) - System Design

Student: Khrisean Stewart

Course: Special Projects

Date: 21 August 2026

Repository: https://github.com/KhriseanStewart/VTDI_Platform

Live app: <https://vtdi-platform.vercel.app>

1. Architecture Overview

OutYah uses a client-server architecture: a React SPA on Vercel communicates with Supabase (Auth, Postgres, Storage) and Google Maps Platform APIs. The browser owns presentation and interaction, while Supabase provides the durable data and security boundary. Context providers keep authentication, catalog data, and the user's saved outing state available across routes without tightly coupling page components.

Browser (React/Vite)

- | - AuthContext / DataContext / AppContext
- | - Public pages + Admin portal
- + -> Supabase Auth + REST (PostgREST) + Storage
- + -> Google Maps JS + Directions

2. Technology Selection

- Frontend: React 19, Vite, React Router, Tailwind CSS v4, Lucide icons
- Backend-as-a-Service: Supabase (Postgres, Auth, RLS, Storage)
- Maps: Google Maps JavaScript API, Places Autocomplete, Directions, Place Details
- Hosting: Vercel (SPA rewrites in vercel.json)
- Tooling: Bun for scripts and local development
- Version control: GitHub

2.1 Selection rationale

React and Vite were selected for fast component-driven development and a lightweight production build. Supabase reduces backend setup while retaining a relational database, row-level security, authentication, and storage. Google Maps supplies reliable map tiles, markers, and routing. Vercel provides automatic deployment from GitHub and supports the single-page rewrite required by React Router.

3. Database Design (ERD description)

Core entities and relationships:

- profiles (1:1 auth.users) - role user|admin
- places - venues with lat/lng, cover/gallery images, hours, rating aggregates
- place_reviews - source-tagged reviews (google|outyah|tripadvisor|yelp|instagram)
- events - optional FK to places; starts_at/ends_at; recurring flags
- posts - Instagram-style media; optional FK to places
- post_comments - FK to posts
- favorites - composite PK (user_id, place_id)
- plan_stops - composite PK (user_id, place_id) + position

```
auth.users --1:1-- profiles
places --1:N-- place_reviews
places --1:N-- events
places --1:N-- posts --1:N-- post_comments
users --N:M-- places (via favorites)
users --N:M-- places (via plan_stops ordered)
```

3.1 Review model

Visitor reviews no longer live only as JSONB on places. The place_reviews table stores author, rating, body, optional business reply, posted_at, and a source tag so the UI can badge Google, OutYah, Tripadvisor, and other origins. A trigger keeps places.review_count aligned with stored rows. Place ratings displayed on cards use Google Place Details

aggregates when synced; listed review snippets come from the Places API (most relevant + newest). Signed-in users may insert OutYah-sourced reviews only.

4. Security Design (RLS)

- Public SELECT on places, events, posts, comments, place_reviews
- Admin ALL on catalog tables via is_admin()
- Authenticated users INSERT OutYah reviews for themselves only
- Users manage only their favorites and plan_stops
- Storage bucket media: public read, admin write

The client uses only the Supabase anon key. Authorization is enforced in PostgreSQL rather than by hiding controls in the browser. Admin screens still check profiles.role for a clear user experience, but RLS remains the final enforcement layer. Database passwords and the service-role key are server-only and excluded from Git.

5. Interface Design

Public and admin navigation share one application shell: desktop sidebar and mobile bottom nav, with admin links nested under Favorites for authorized users. Home uses a Feed/Map segmented control and a places grid with cover imagery. Venue detail tabs include Overview, Reviews (source-filtered), Instagram posts, and Hours. Empty states provide clear CTAs when catalogs are empty.

5.1 Responsive behavior

- Desktop: persistent left navigation and multi-column content grids.
- Mobile: fixed bottom navigation, horizontally scrollable chips, and single-column forms.
- Feed and place cards emphasize photography; Tailwind utility recipes live in src/lib/ui.js.
- Admin forms use parish dropdowns, Google Places location pickers, and media uploads.

5.2 Data-state behavior

Every catalog screen distinguishes loading, populated, empty, and error states. Empty states explain what is missing and provide an appropriate action, such as adding the first place for an administrator or returning to discovery for a regular user.

6. Key UML-style Flows

6.1 Discovery to Plan

1. User opens Feed, filters/search posts
2. Opens Venue Detail
3. Favorites and/or Add to outing
4. Planner shows ordered stops + route map
5. Get Directions opens Google Maps URL

6.2 Admin content publish

1. Admin signs in
2. profiles.role verified as admin
3. CRUD place/event/post in /admin
4. Public DataContext refresh shows content

7. Deployment Design

- Build: bun/vite -> dist/

- Vercel env: VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_GOOGLE_MAPS_API_KEY
- DB scripts: supabase/migrations/001_init.sql, 002_events_schedule.sql, 003_place_reviews.sql
- Optional: supabase/full_setup.sql; seed helpers under scripts/
- Client never receives service_role key

8. Module Responsibilities

- AuthContext: session lifecycle, profile lookup, sign-in, sign-up, sign-out, admin role.
- DataContext: Supabase catalog queries, refresh, and in-memory lookup helpers.
- AppContext: favorites and ordered plan state, including per-user persistence.
- lib/data.js: database-row mapping, place/event/post queries, place review CRUD.
- lib/reviews.js: review source metadata, mapping, and rating helpers.
- PlaceReviews: source filters, Google/OutYah list, signed-in submit form.
- pages/admin: auth-gated CRUD nested in the main shell (AdminLayout is gate only).
- scripts/seed_jamaica_catalog.mjs: island-wide places + 6-month events upsert.
- scripts/seed_jamaica_island_photos.mjs: Google photos for island venues.
- scripts/seed_real_place_reviews.mjs: sync Google Place Details reviews into place_reviews.

9. External integrations

- Google Maps JS: map markers, venue map, multi-stop directions URLs.
- Google Places Autocomplete: admin location picker (address, lat/lng).
- Google Place Details: real venue ratings and review text for seeding/sync.
- Supabase Storage media bucket: place cover and gallery uploads.
- Instagram Graph: optional; tokens may be empty - posts remain admin-managed.